DCS
UNIVERSITY OF JAFFNA

Introduction to Support Vector Machines (SVMs)

Academic Year: 2022/2023

Support Vector Machine (SVM)

- Introduced in 1992
- Developed by Vladimir N. Vapnik and co-workers
- An optimally defined surface
- Typically non-linear in the input space
- Linear in a higher dimensional feature space
- Implicitly defined by a kernel function
- SVM is a practical learning method based on Statistical Learning Theory

SVMs are used for:

- Classification
- Regression and Data Fitting

Idea of SVMs

It maps the input vectors $\underline{x}$ into a high-dimensional feature space Z through some non-linear mapping, chosen a priori. In this space, an optimal separating hyperplane is constructed.

Idea of SVMs …

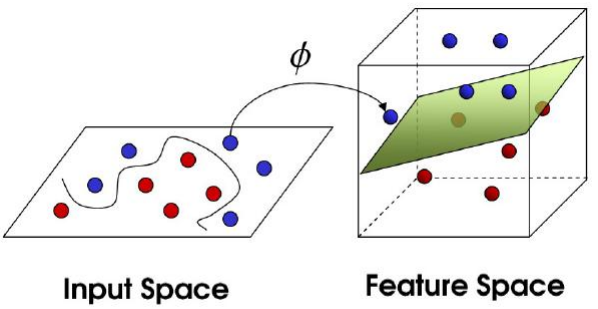


Image from: Interactive Image Segmentation using Kernel Methods, Olivier Duchenne, ENS Paris; olivier.duchenne@ens.fr

Classification

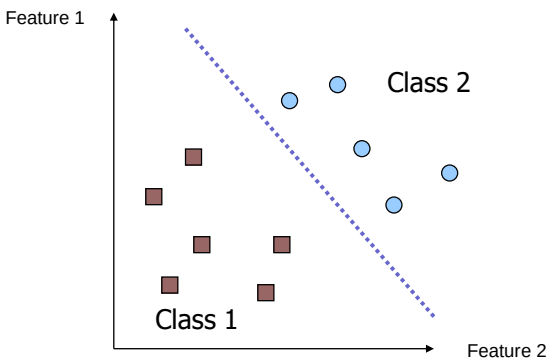
- Learn from the features and find a function that correctly classifies the patterns in the training data. This is called *generalisation*.

$$(\mathbf{x}, y) \longrightarrow \boxed{f} \longrightarrow \hat{y} = f(\mathbf{x})$$

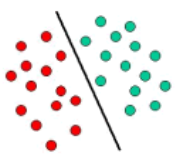
- Produce a classifier f that minimises the expected loss.

Binary-Class Classifier

- classifies data into two classes

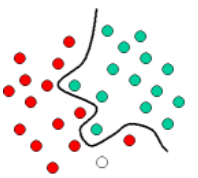


(1). Separable Case



Linear line decision boundary

(Linearly separable)

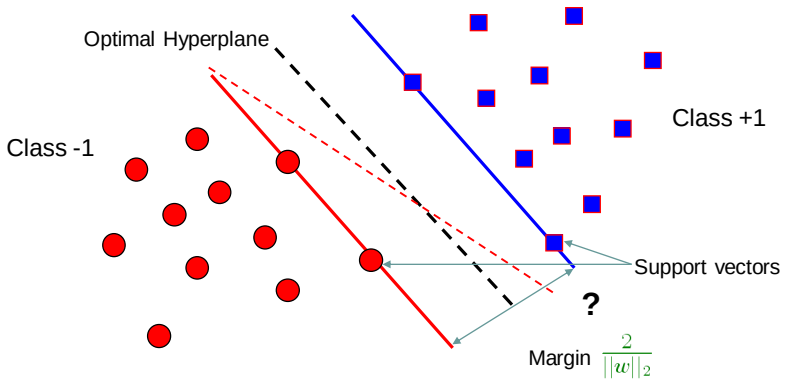


Complex curve decision boundary

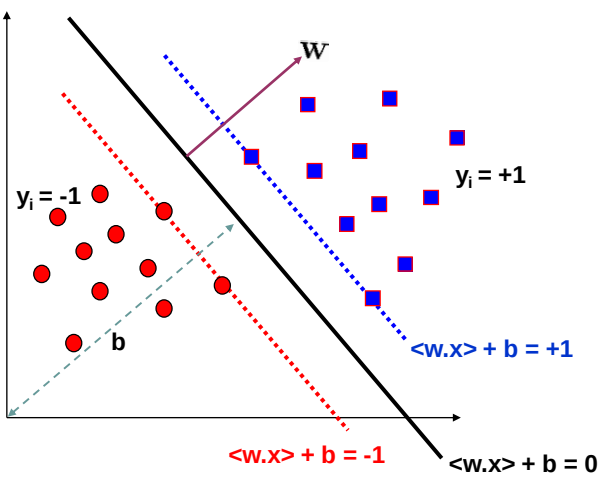
(Non-linearly separable)

Optimal Hyperplane

Learning to find the plane with largest margin to separate the two classes.



Optimal Hyperplane …..



Hyperplane …

- Canonical form of hyperplane

  $<w.x_i> + b \geq 1$ if $y_i = +1$

  $<w.x_i> + b \leq -1$ if $y_i = -1$

- Compact notation for these inequalities

  $y_i[(w.x_i) + b] \geq 1$ $i=1,2,\ldots,\ell$

Hyperplane …

- The margin of a set of points $\{x_1,x_2,\ldots,x_m\}$ is defined as the margin of the point closest to the hyperplane.

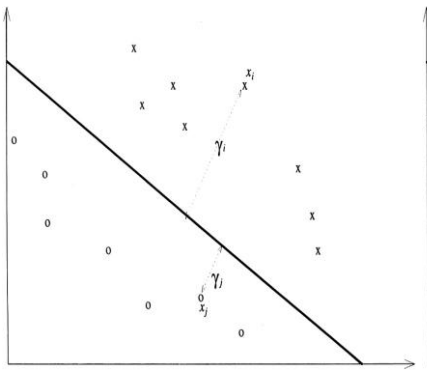
$$\gamma = \max_{1\leq i\leq m} \gamma_i = \min_{1\leq i\leq m} \frac{|< w.x_i > +b|}{\|w\|}$$

where,

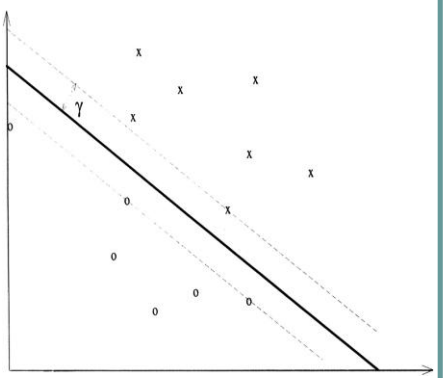
γ - margin of training set

γ_i - geometric margin

Hyperplane …



The geometric margin of two points



The margin of a training set

The decision boundary should be as far away from the data of both classes as possible, thus should maximize the margin.

Images from: http://www.cs.wm.edu/~stallman/Intro%20to%20SVM/images/

Finding the Decision Boundary

- Let $\{x_1, ..., x_n\}$ be our data set and let $y_i \in \{1,-1\}$ be the class label of x_i
- The decision boundary that classifies all points correctly can be found by solving the following constrained optimization problem

$$\text{Minimize } \frac{1}{2}||\mathbf{w}||^2$$

$$\text{subject to } y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 \qquad \forall i$$

- This is a constrained optimization problem.

Minimization Method for Constrained Optimization

- The minimization can be solved by the *saddle point* of the Lagrange functional (Lagrangian):

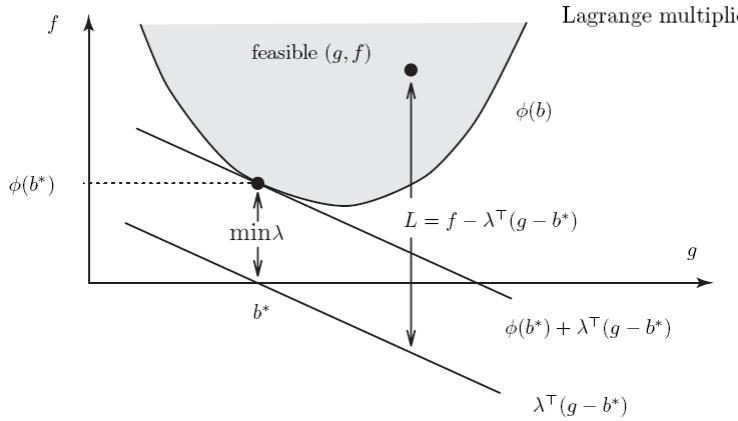
$$L(w,b,\alpha) = \frac{1}{2} < w.w > - \sum_{i=1}^{l} \alpha_i \{[(w.x)+b]\, y_i - 1\}$$

Where α_i are Lagrange multipliers. The Lagrangian has to be minimized w.r.t w,b and maximized w.r.t $\alpha_i > 0$

Lagrangian Method for Constrained Optimization

Lagrangian: $L(x,\lambda) = f(x) + \lambda(b - g(x))$

f is to be maximized
constraint $g(x) = b$
Lagrange multipliers λ



Reference: http://nes.aueb.gr/courses/ecobiz/Lagrange.pdf

Decision Function

- To find the class of a particular vector x, we use the decision function:

$$f(x) = sign(< w.x > + b)$$

- The dual representation of the decision function:

$$f(x) = sign(\sum_{i=1}^{l} y_i \alpha_i < x.x_i > + b)$$

Problem in Linear Learning Machines

- Linear learning machines (such as the hyperplane classifier) have limited computational power.
- In general, complex real-world applications require learning machine with more expressive power.

Proposed Solutions:

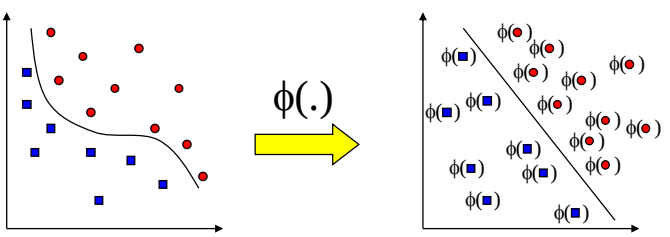
- **Solution-1:** create a network of simple linear classifiers (in the form of neural networks) and thus be able to represent nonlinear decision surfaces.

☹ **However,**

- Neural networks have a number of inherent problems [local minima, many tunable parameters].
- Very complex to analyse a neural network mathematically.

Proposed Solutions …

- **Solution-2:** map the input vectors into a feature space where they are linearly separable using a non-linear mapping Φ



Input space Feature space

Image from: Lecture Notes of CSE 802, Martin Law, Department of Computer Science and Engineering, Michigan State University

Proposed Solutions …

Decision function: $f(x) = sign(\sum_{i=1}^{l} y_i \alpha_i < \phi(x).\phi(x_i) > +b)$

☹ *However,*

- Feature space will translate to $<\phi(x).\phi(y)>$ clearly very *computationally expensive*, especially if the mapping is to a high-dimensional space.

- For this reason we may use Kernel functions K(x,y) that given two vectors in input space, returns the dot product of their images in feature space such that, K(x,y) = $<\phi(x).\phi(y)>$

Kernel

A kernel is an interface between the data and the learning machine that defines the mapping into the feature space.

Examples of Kernels:

- Polynomial Kernel $k(\vec{x}, y) = ((\vec{x} \cdot y) + 1)^d$
- Radial Basis (Gaussian) Kernel $k(\vec{x}, y) = \exp(-\|\vec{x} - y\|^2 / (\sigma^2))$
- Neural Network Kernel (Sigmoid Kernel) $k(\vec{x}, y) = \tanh(\kappa(\vec{x} \cdot y) + \Theta)$

Mercer Theorem: guarantees the dot product in the equation can be substituted by kernel function

SVM & Kernel….

Decision function: $f(x) = sign(\sum_{i=1}^{l} y_i \alpha_i K(x, x_i) + b)$

SVM are nothing more (or less) than linear learning machines expressed in dual form that map their input vectors to a feature space by the use of kernels and compute the optimal hyperplane there.

(2). Nonseparable Case

- In order to deal with this case, the soft margin SVM is suggested [Cortes and Vapnik, 1995].
- A new set of non-negative variables ξ_i is introduced to measure the violation of the constraints.

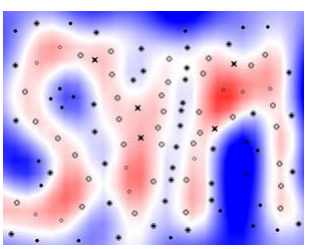
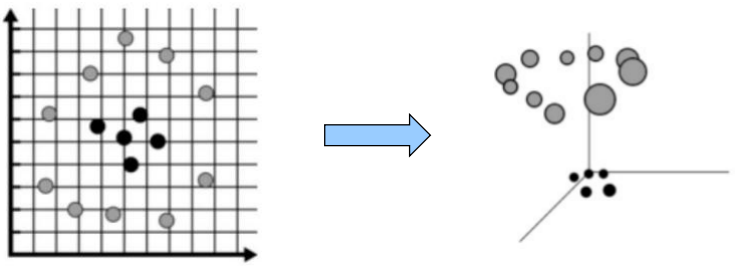
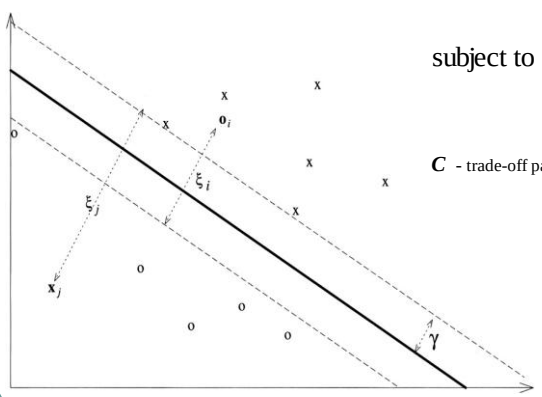


Image from: Daniel Whiteson and Axel Naumann, http://www-d0.fnal.gov/~danielw/svm/

Non-Linear SVMs



Nonseparable Case …



Minimize $\frac{1}{2}\|w\|^2 + C\sum_{i=1}^{l}\xi_i$

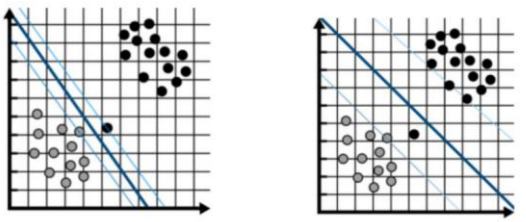
subject to $y_i(< w.x_i > +b) \geq 1 - \xi_i$

$\xi_i \geq 0;\ i = 1,2,\ldots,l$

C - trade-off parameter between error and margin

Image from: http://www.cs.wm.edu/~stallman/Intro%20to%20SVM/images/

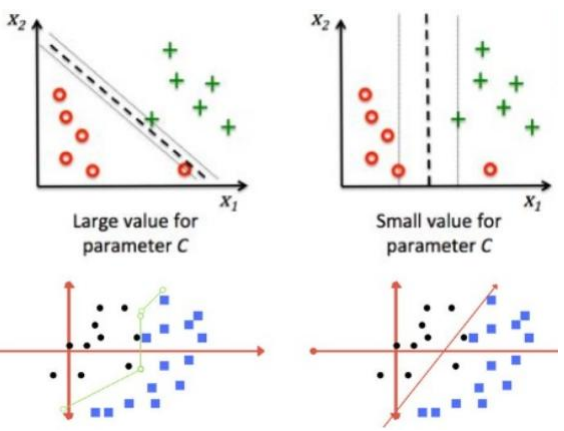
SVMs …



C = 1,000 C = 0.001

- Cost (**C**): When picking the support vectors, the "cost" of incorrectly classifying a data point.
- Higher C values means that there is a higher cost to incorrectly classifying a training point. Too high means we'll underfit.
- Lower C values means that there is a lower cost to incorrectly classifying a training point. Too low means we'll overfit.

Regularisation



Expected Loss in Classification

- Having the generalized Optimal hyperplane, the expectation value of the probability of committing an error on a test example:

$$\mathbf{E[P(error)]} \leq \frac{\mathbf{E[number\ of\ support\ vectors]}}{\mathbf{(number\ of\ training\ vectors)-1}}$$

- This bound depends neither on the dimensionality of the space, nor on the norm of the vector of coefficients, nor on the bound of the norm of the input vectors

Kernel

- Pros

- It works really well with clear margin of separation
- It is effective in high dimensional spaces.
- It is effective in cases where number of dimensions is greater than the number of samples.
- It uses a subset of training points in the decision function (called support vectors), so it is also memory efficient.

Kernel …

- Cons

- It doesn't perform well, when we have large data set because the required training time is higher
- It also doesn't perform very well, when the data set has more noise i.e. target classes are overlapping
- SVM doesn't directly provide probability estimates, these are calculated using an expensive five-fold cross-validation. It is related SVC method of Python scikit-learn library.

Multiclass Classifier

- Each training point belongs to one of the different classes.
- The goal is to construct a function which, given a new data point, will correctly predict the class to which the new point belongs.

Multiclass SVMs

Two approaches to solve multi-class pattern recognition problems using SVMs

One-vs-All

- +ve training data in the +1 class
  -ve training data in the rest
- N binary SVMs needed
  (N - the number of the classes)

One-vs-One

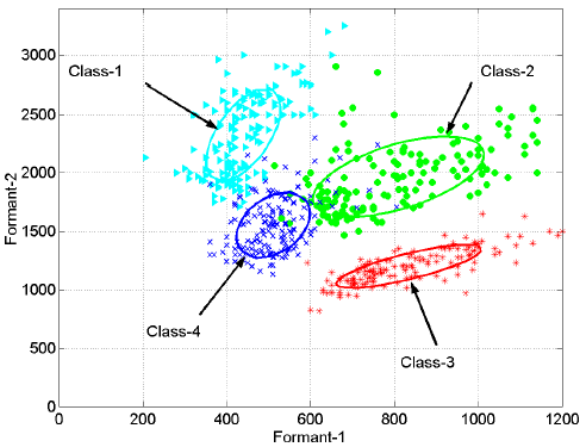
- +ve training data in the +1 class
  -ve training data in the other class
- $\frac{1}{2}$N(N-1) binary SVMs needed

SVM-based multiclass classifier

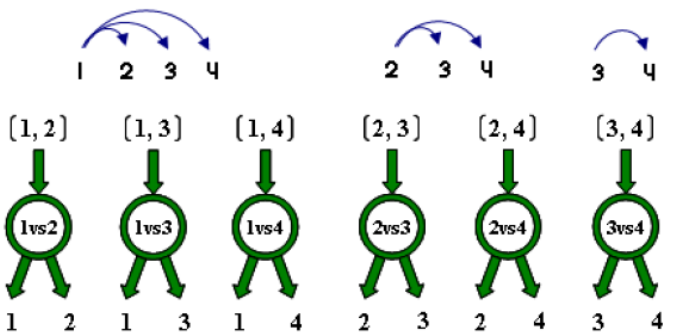
There are three standard techniques frequently employed by SVMs to tackle multiclass problems, namely one-versus-one (OVO), one-versus-all (OVA), directed acyclic graph (DAG), and unbalanced decision tree (UDT).

To illustrate those standard techniques diagrammatically, the following four classes (i.e. *N* =4) are selected from the Peterson and Barney (1952)'s dataset, consisting of 10 classes that are vowel sounds characterised by the first two formant frequencies. A circle in the diagram denotes a binary SVM classifier.

SVM-based multiclass classifier …



(i) One-versus-one (OVO)

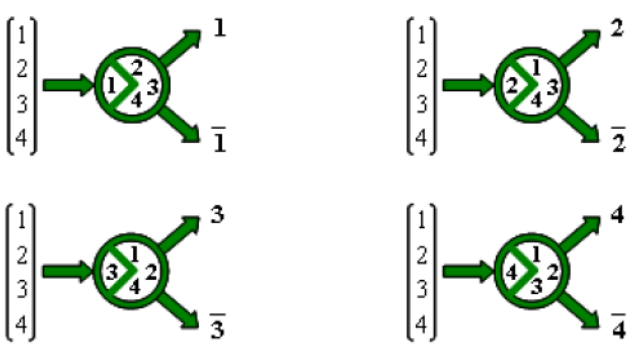


One-versus-one (OVO)

OVO method is implemented using a “max-wins” voting strategy. This method constructs one binary classifier for every pair of distinct classes, in total it constructs *N(N-1)/2* binary classifiers, where *N* is the number of classes. The binary classifier Cij is trained with examples from the i*th* class and j*th* class only,

where examples from class *i* take positive labels while examples from class *j* generally take negative labels. For an example *x*, if classifier Cij predicts *x* is in class *i*, then the vote for class *i* is increased by one, otherwise the vote for class *j* is increased by one. The max-wins strategy then assigns *x* to the class receiving the highest voting score.

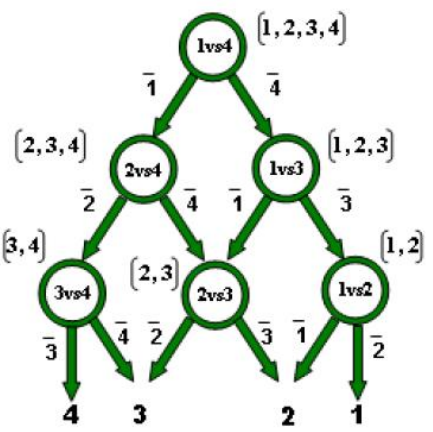
(ii) One-versus-all (OVA)



One-versus-all (OVA)

OVA method is implemented using a “winner-takes-all” strategy. It constructs *N* binary classifier models. The i*th* binary classifier is trained with all the examples in the i*th* class with positive labels, and the examples from all other classes with negative labels. For an example *x*, the winner-takes-all strategy assigns it to the class with the highest classification boundary function value.

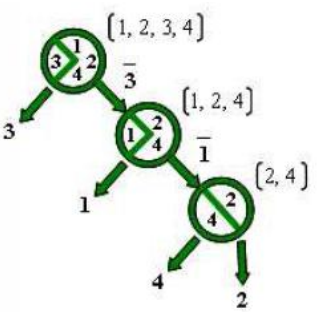
(iii) Directed acyclic graph (DAG)



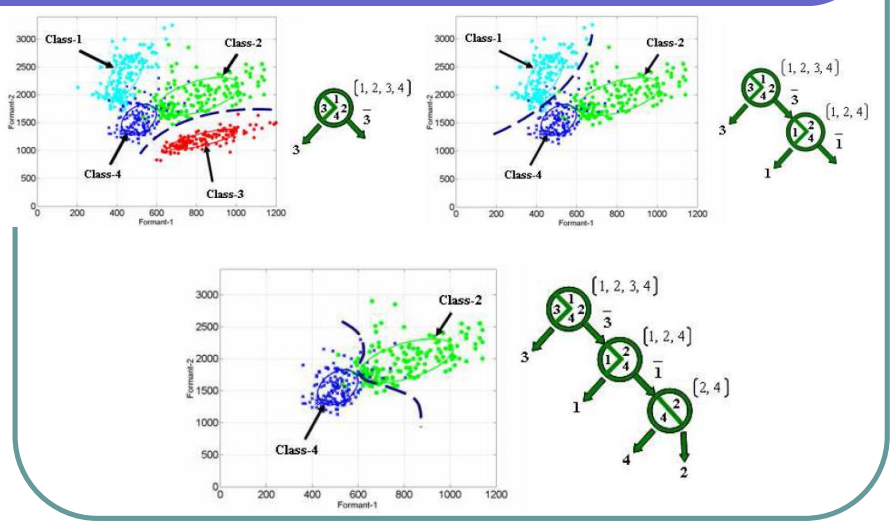
Directed acyclic graph (DAG)

DAG-based SVMs are implemented using a “leave-one-out” strategy. The training phase of the DAG is the same as the OVO method, solving *N(N-1)/2* binary classifiers. In the testing phase it uses a rooted binary directed acyclic graph which has *N(N-1)/2* internal nodes and *N* leaves. Each node is a classifier Cij from OVO. An example *x* is evaluated at the root node and then it moves either to the left or the right depending on the output value.

(iv) Unbalanced decision tree (UDT)



Unbalanced decision tree (UDT)

UDT-based SVMs are implemented using a “knock-out” strategy with at most *(N-1)* classifiers to make a decision on any input pattern. Each decision node of UDT is an optimal classification model. The optimal model for each decision node is the OVA-based classifier that yields the highest performance measure. Starting at the root node, one selected class is evaluated against the rest by the optimal model. Then the UDT proceeds to the next level by eliminating the selected class from the previous level of the decision tree. The UDT terminates when it returns an output pattern at a level of the decision node, while DAG needs to proceed until a leaf node is reached.

Unbalanced decision tree (UDT)